

推薦序一

收到 Kevin 邀請的時候，我第一個念頭其實是：在這個時代，寫書要有點勇氣，甚至連看書感覺都有點門檻。還不用到 AI 時代，從網路時代開始，大家就是上網學東西，Stack Overflow 看一看，就可以自己搞定了。如果喜歡分享，社群網路其實也很夠了，在上面可以回饋社群，甚至影響世界。在這種情況下，看書、寫書似乎變成一種奢侈且沒效率的做法。

不過看了這本書的目錄之後，的確有吸引到我去細看內容。身為一位喜歡技術的工程師，我覺得這本書的內容很完整，甚至有點「太」完整了！前面講 LLM 的基礎，接著講怎麼運用框架實作 Agent，然後到 RAG，最後還有資安。那我為什麼會想看？這些技術我都會，或是都可以自己上網查。我最後的結論是，我想知道其他工程師是怎麼看這些技術。

例如，Kevin 選擇從 LLM 的基礎開始寫。現在其實沒什麼人在訓練模型了，大家都是在寫包住 LLM 的那些 Agent Harness。但這本書還是決定從頭講，Kevin 將其視為一個程式的典範轉移：以前軟體工程的規則非常清楚，而現在 LLM 代表著一個超強但有很大不確定性的功能，你得知道它能與不能，才能正確處理那些過去不曾面對的問題。另一個例子是最後提到的資安，很多 Agent 實作的教學應該不會提到這一點，畢竟它就是另一個面向。但 Kevin 之前也寫過資安相關的文章，表示他人本覺得這是軟體工程的一部分，一個產品的安全性應該內化在開發階段，而不是最後上線前再來弱掃什麼的。

這個選擇是一位資深工程師的 taste，中文應該翻成「品味」。所以，建議在看這本書的時候，除了實際跟著做一遍之外，也可以去想想作者為什麼決定要講這個？為什麼選擇這樣做？

我覺得現在大家的產出會更快，AI 時代講講話就可以產出，實際上結果也不會差太多。菜鳥工程師會覺得自己很厲害，可以很快做出東西，但是他們變成資深工程師中間那段摸索的過程消失了，而這段痛苦的过程有可能是成長的關鍵。

試想不知道 Time Complexity 的人怎麼會下 prompt 要求 AI 注意？資安有那麼多面向，的確 AI 聰明到用白話講「請避免 SQL Injection」就可以了，但如果連 SQL Injection 這個詞都不知道怎麼辦？叫 AI 做一個 Agent，它一定做得出來，用起來也不會差太多，但那些細節不自己走過一遍，就不會知道中間會有什麼雷。追根究底還是得增加自己的知識極限，才能要求 AI 去提升產出品質。

這也許是書本這個比較慢速的媒介，在快時代裡珍貴的地方。在其他快節奏的媒介下，一般人很難有機會去思考這些事。當然，因為需要時間，所以要好好挑書，或是該說好好挑一位個有經驗的工程師。Kevin 從新創出來的，有那種一個人負責一整個產品的經驗，可以自己寫出蠻耐看的前端，一路到後端甚至資安、部署。你要找到一個扎實的工程師，而且他還願意分享，這件事本身就很難得。

總之，如果你拿到這本書，記得不要只是照做，多去思考作者的選擇邏輯。要知道半年、一年後技術可能就變了，書裡的具體做法可能就過時了。但如果你學到的是一位資深工程師怎麼看待新技術、怎麼做選擇，那這個能力不會過期。

張天豪

永豐金控科技長

推薦序二

我在科技業待了將近二十年，前十年在大型科技公司，後十年自己和學長創業。這段經歷讓我養成一個習慣：判斷一個人，不看他說了什麼，而是看他做了什麼。

我跟雋凱的第一次合作大概可以追溯到八年多前在潮網共事。當時對他最深的印象，到現在都沒有變過，你跟他講一個想法，他就可以很快先做出一個能動的東西，然後再來談下一步。

後來我到永豐金控數位科技處任職，再次有機會跟他共事。場景換了，但那個特質完全一樣。他現在負責的是快速將概念轉化為可驗證的原型，協助推動全集團的工作效率。很多時候，一個想法從出現到變成能展示的系統，中間需要的不只是寫程式的能力，更是一種把模糊的東西變成具體產出的本事。從大學時期頻繁參加實作競賽拿到名次，再兩次在新創負責開發新產品，最後到企業裡做 PoC，走的其實一直都是同一條路，即先建立 MVP，讓東西跑起來，再一步步優化。

聽到他在寫書並邀請我寫推薦序，說實話有點訝異——沒想到一個技術工程人員，能把這麼複雜的實作過程，一步一步講述得那麼清楚。

這不是一本「介紹 AI 有多厲害」的書。它在做的事情，是告訴你怎麼把 LLM 變成一個可以放進系統裡、可以維運、可以信任的元件。

這本書從第一個聊天機器人開始，一路走過串流回應、多輪對話、工具呼叫、流程編排、檢索增強生成，再到多代理協作與人機互動設計，最後帶到本地部署與資安防護。這是一條從一開始「能跑」走到最後「能上線」的完整路徑，而不只是把技術點攤開來的知識拼盤。

書裡同一個範例會從最簡單的版本開始，反覆迭代好幾次，每次只加一層能力。讀起來不只是學到最終的寫法，更清楚看到每一層「為什麼被需要」。這也跟過去在新創做產品的經驗很像，先有最小可行的東西，碰到問題再長出功能，而不是一開始就設計一個龐然大物。

另一個讓我感到有趣的部分，是書中花了相當大的篇幅談「什麼時候該讓人介入」。在 AI Agent 的討論裡，多數人急著往全自動化走，但這本書很清楚地指出，成熟的系統設計，是知道哪些環節必須保留人的判斷。這個設計觀念對任何把 AI 放進生產環境的團隊來說，都是必修課。

最後一章的資安設計則是我覺得最特別的地方。從語言層、資料層到行為層，完整拆解風險如何在系統中串連與放大，再給出多層防禦的做法。一般大多技術人員會聚焦在絢麗的技術能力，但這本書則是花了不少的篇幅把安全議題也寫了進來，很少見。

整本書讀下來，字裡行間透露的不是「我知道很多東西」，而是「我踩過這些坑，所以知道你會需要這些」。什麼時候該用框架、什麼時候該自己寫、什麼時候該信任模型、什麼時候該加上防護，這些判斷背後的工程直覺，正是這本書有價值的地方。

我在永豐帶的是數位科技的團隊，團隊裡有這樣一個人，能把前沿技術快速轉化成實際產出，同時還願意花時間把經驗整理成書讓更多人受益，這件事本身就說明了一些東西。我們做的事情不只空談，而是把東西做出來。這本書也是同一個邏輯。

如果你想認真把生成式 AI 落地到產品或企業系統裡的開發者，這本書會給你一條走得通的路。它不講華麗的願景，但交出來的東西是真的能用的，跟寫這本書的人一樣。

周立夫

永豐金控數位科技處副處長